

Database Connectivity with Python and MariaDB (and Other Databases)

1. Introduction

Database connectivity is the process of establishing communication between a Python application and a Database Management System (DBMS) to perform operations such as:

- Creating databases
- Creating tables
- Inserting records
- Retrieving records
- Updating records
- Deleting records
- Performing transactions

Database connectivity forms the backbone of:

- Web Applications
- Enterprise Applications
- Banking Systems
- E-Commerce Platforms
- Data Science Applications
- Cyber Security Monitoring Systems
- Digital Forensics Management Systems

2. Why Database Connectivity?

Consider a Student Management System.

Without a database:

```
students = [  
    ["John",85],  
    ["Alice",90]  
]
```

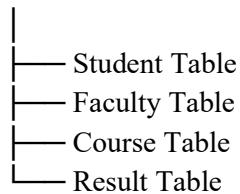
Problems:

- Data lost after program termination

- Difficult to manage large data
- No security
- No concurrent access

With Database:

Database



Benefits:

- Permanent storage
- Fast retrieval
- Data integrity
- Multi-user access

3. Python Database API (DB-API)

Python provides a standard interface called:

Python DB-API 2.0

Defined in:

PEP 249

Common DB-API functions:

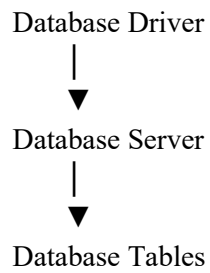
```

connect()
cursor()
execute()
fetchone()
fetchall()
commit()
rollback()
close()
  
```

4. Database Connectivity Architecture

Python Application





Example:

```
Python
|
mysql.connector
|
MariaDB
```

5. Popular Databases Used with Python

Database	Type
MariaDB	Relational
MySQL	Relational
PostgreSQL	Relational
SQLite	Relational
Oracle Database	Relational
Microsoft SQL Server	Relational
MongoDB	NoSQL

6. Database Drivers

A driver acts as a bridge between Python and the database.

Database	Python Driver
MariaDB	mariadb
MySQL	mysql-connector-python
PostgreSQL	psycopg2
SQLite	sqlite3
Oracle	cx_Oracle

SQL Server	pyodbc
MongoDB	pymongo

7. MariaDB and Python Connectivity

What is MariaDB?

MariaDB is an open-source relational database derived from MySQL.

Features:

- Open Source
- High Performance
- Secure
- ACID Compliant
- Enterprise Ready

Installing MariaDB Connector

pip install mariadb

Importing MariaDB Library

import mariadb

Establishing Connection

```
import mariadb

conn = mariadb.connect(
    host="localhost",
    user="root",
    password="password",
    database="studentdb"
)

print("Connected")
```

Output:

```
Connected
```

Creating Cursor

A cursor executes SQL queries.

```
cursor = conn.cursor()
```

Creating Database

```
cursor.execute(
    """
    CREATE DATABASE studentdb
    """
)
```

Creating Table

```
cursor.execute("""
CREATE TABLE students(
    id INT PRIMARY KEY,
    name VARCHAR(50),
    marks INT
)
""")
```

Inserting Data

```
sql = """
INSERT INTO students
VALUES
(1,'John',85)
"""

cursor.execute(sql)

conn.commit()
```

Reading Data

```
cursor.execute(
    "SELECT * FROM students"
)

rows = cursor.fetchall()
```

```
for row in rows:  
    print(row)
```

Output:

```
(1, 'John', 85)
```

Updating Records

```
cursor.execute(  
    """  
    UPDATE students  
    SET marks=90  
    WHERE id=1  
    """  
)  
  
conn.commit()
```

Deleting Records

```
cursor.execute(  
    """  
    DELETE FROM students  
    WHERE id=1  
    """  
)  
  
conn.commit()
```

Closing Connection

```
cursor.close()  
  
conn.close()
```

Complete MariaDB Example

```
import mariadb  
  
conn = mariadb.connect(  
    host="localhost",  
    user="root",  
    password="password",  
    database="studentdb"  
)
```

```
cursor = conn.cursor()

cursor.execute("""
INSERT INTO students
VALUES (2,'Alice',92)
""")

conn.commit()

cursor.execute(
"SELECT * FROM students"
)

for row in cursor.fetchall():
    print(row)

cursor.close()
conn.close()
```

8. MySQL Connectivity

Install connector:

```
pip install mysql-connector-python
```

Connection:

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="password",
    database="studentdb"
)
```

Cursor:

```
cursor = conn.cursor()
```

Execute query:

```
cursor.execute(
"SELECT * FROM students"
)
```

9. SQLite Connectivity

What is SQLite?

SQLite is a lightweight database built into Python.

No installation required.

Import SQLite

```
import sqlite3
```

Create Database

```
conn = sqlite3.connect(
    "student.db"
)
```

Creates:

```
student.db
```

Create Table

```
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE students(
    id INTEGER,
    name TEXT,
    marks INTEGER
)
""")
```

Insert Record

```
cursor.execute("""
INSERT INTO students
VALUES
(1,'John',85)
""")

conn.commit()
```

Retrieve Data

```
cursor.execute(
"SELECT * FROM students"
)
```

```
print(cursor.fetchall())
```

Output:

```
[(1, 'John', 85)]
```

Advantages of SQLite

- Built into Python
- Lightweight
- No server required
- Easy deployment

10. PostgreSQL Connectivity

Install driver:

```
pip install psycopg2
```

Connection:

```
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    database="studentdb",
    user="postgres",
    password="password"
)
```

11. MongoDB Connectivity

What is MongoDB?

MongoDB stores data as documents instead of tables.

Install Driver

```
pip install pymongo
```

Connect to MongoDB

```
from pymongo import MongoClient

client = MongoClient(
    "mongodb://localhost:27017/"
)

db = client["studentdb"]
```

Insert Document

```
collection = db["students"]

collection.insert_one({
    "name": "John",
    "marks": 85
})
```

Retrieve Documents

```
for doc in collection.find():
    print(doc)
```

12. Parameterized Queries

Avoid SQL Injection.

✗ Unsafe:

```
name = input()

query = f"""
SELECT * FROM students
WHERE name='{name}'
"""
```

✓ Safe:

```
cursor.execute(
    """
SELECT * FROM students
WHERE name=?
""",
    (name,)
)
```

For MariaDB/MySQL:

```
cursor.execute(
    """
    SELECT * FROM students
    WHERE name=%s
    """,
    (name,)
)
```

13. Transactions

A transaction ensures data integrity.

Example:

```
try:

    cursor.execute(query1)

    cursor.execute(query2)

    conn.commit()

except:

    conn.rollback()
```

ACID Properties

Atomicity

All operations succeed or none.

Consistency

Database remains valid.

Isolation

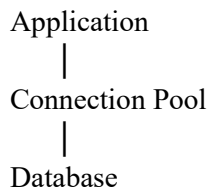
Transactions remain independent.

Durability

Changes are permanent.

14. Connection Pooling

Instead of creating new connections repeatedly:



Benefits:

- Faster performance
- Reduced overhead
- Better scalability

15. Database Connectivity in Web Applications

Example using Flask:

```
from flask import Flask
import sqlite3

app = Flask(__name__)

@app.route("/students")

def students():

    conn = sqlite3.connect(
        "student.db"
    )

    return "Connected"
```

16. Database Connectivity in Data Science

```
import pandas as pd
import sqlite3

conn = sqlite3.connect(
    "student.db"
)

df = pd.read_sql_query(
    "SELECT * FROM students",
```

```
conn
)

print(df)
```

Output:

```
id name marks
0 1 John 85
```

17. Database Connectivity in Cyber Security

Applications:

- Log Management
- Security Event Storage
- Threat Intelligence
- SIEM Systems

Example:

```
cursor.execute("""
INSERT INTO security_logs
VALUES(
'Failed Login',
CURRENT_TIMESTAMP
)
""")
```

18. Database Connectivity in Digital Forensics

Applications:

- Evidence Storage
- Chain of Custody Tracking
- Timeline Analysis

Example:

```
cursor.execute("""
INSERT INTO evidence
VALUES(
101,
'memory_dump.raw'
)
""")
```

MariaDB vs MySQL vs PostgreSQL vs SQLite

Feature	MariaDB	MySQL	PostgreSQL	SQLite
Open Source	Yes	Yes	Yes	Yes
Server Required	Yes	Yes	Yes	No
Performance	High	High	Very High	Moderate
Scalability	High	High	Very High	Low
Enterprise Features	Good	Good	Excellent	Limited
Embedded Usage	No	No	No	Yes

Best Practices

1. Always close connections.
2. Use parameterized queries.
3. Handle exceptions properly.
4. Use transactions.
5. Encrypt sensitive data.
6. Implement connection pooling.
7. Follow least-privilege access.
8. Validate all user inputs.

Summary

Database Connectivity in Python enables applications to communicate with databases using specialized drivers and the Python DB-API. Python supports a wide variety of databases including MariaDB, MySQL, PostgreSQL, SQLite, and MongoDB. Through connections, cursors, SQL queries, transactions, and connection management, Python applications can efficiently store, retrieve, update, and analyze data for enterprise systems, web applications, AI platforms, cyber security solutions, and digital forensics environments.